

CS–C++ Foundations: Homework #1

(Based on Lectures 1–2)

Problem 1 – Hello & Echo (I/O warm-up)

Write a program that:

- Prints Hello, world! (exactly) and a newline.
- Then prompts the user with What is your full name? and reads:
 - one version using `std::cin » name` (store into a `std::string`),
 - another version using `std::getline`.
- Print back what was captured in each version on separate lines so you can observe the difference.

Problem 2 – Variables & Types mini-report

Write a program that declares one variable of each of the following: `char`, `bool`, `short`, `int`, `long`, `long long`, `float`, `double`, `long double`. For each, print:

- `sizeof(type)` in bytes,
- for integral types: whether the type is treated as signed by default (explain in comments),
- for `char`: also print its numeric (ASCII) code when initialized to 'A' and then after `+= 25`.

Problem 3 – Scope probe

Create a file `scope_probe.cpp` that demonstrates:

- a global variable `g`,
- a local variable `g` inside `main` that *shadows* the global one,
- an inner block that declares another `g` and prints values at each nesting level to show which one is visible.

Add one commented line that *would not compile* because a variable is out of scope—explain why.

Problem 4 – Integer overflow lab

- With unsigned `int` variables `x=0`, compute `y = x - 1` and `z = y + 1`. Print `x`, `y`, `z`.
- With unsigned `int a = 123456`, print `a * a`. Discuss in comments why this result wraps modulo a power of two.

- c) Repeat (a) and (b) with `int` and explain in comments why signed overflow is *undefined behavior*; do *not* rely on any particular printed value.

Problem 5 — Safe average (types & casting)

Read two integers `a` and `b` from standard input. Compute:

- a) integer quotient and remainder of `a / b` and `a % b`,
- b) the precise average of `a` and `b` as a `double`, using correct casting so that no integer division occurs before conversion.

Theoretical Quiz A — Short answer

Answer concisely (1–3 sentences each).

1. What is the role of the `main` function in a C++ program, and what does returning 0 usually indicate?

The `main` function is the entry point of a C++ program where execution begins. Returning 0 generally signals that the program ran successfully without encountering any errors or abnormal termination.

2. Why might `std::getline` capture more input than `std::cin >> name` when reading a user's name?

`std::getline` reads an entire line of input, including spaces and special characters, whereas `std::cin >> name` stops reading when it encounters whitespace, which can result in only part of the input being captured.

3. What is variable *scope*? Give an example of block scope vs. function scope.

Variable scope defines the region of a program where a variable is accessible. For example, a variable declared inside `{ }` has block scope and is only visible within that block, while a variable declared in a function has function scope and can be accessed anywhere inside that function.

4. Why must local variables of simple types (like `int`) be initialized explicitly before reading their value?

Local variables of simple types, such as `int` or `float`, must be explicitly initialized before use because they are not automatically set to a default value. Using an uninitialized variable can lead to undefined behavior since it may contain garbage data left in memory.

5. When should you prefer `double` over `float`?

`Double` should be used instead of `float` when higher precision is needed. Using `double` reduces the risk of rounding errors and allows for more accurate results in computations that require decimal accuracy.

Theoretical Quiz B – Multiple choice

Choose the single best answer.

1. The expression `7 / 3` (both `int`) evaluates to:
 - a) 2.3333
 - b) 2
 - c) 3
 - d) undefined behavior
2. For `unsigned int ui`, which statement is correct?
 - a) Overflow is undefined behavior and may crash.
 - b) Overflow wraps around modulo 2^n for that width.
 - c) Overflow saturates at the max value.
 - d) Compilers always warn and refuse to compile.
3. Which of the following *best* explains what `sizeof(T)` returns?
 - a) The number of decimal digits required to print values of T.

- b) The number of elements T can hold.
- c) The number of bytes occupied by an object of type T on the target platform.
- d) The maximum value of type T.

4. In

```
int x = 1;
{
    int x = 2;
    std::cout << x << "\n";
}
std::cout << x << "\n";
```

the two printed values are:

- a) 1 then 2
 - b) 2 then 1
 - c) 2 then 2
 - d) 1 then 1
5. Which line *correctly* forces floating-point division for int a, b;?
- a) double q = a / b;
 - b) double q = (double)(a / b);
 - c) double q = static_cast<double>(a) / b;
 - d) double q = a / (b * 1);